

Logic-Based Verification of Task Allocation for LLM-Enabled Multi-Agent Manufacturing Systems

Jonghan Lim¹, Mostafa Tavakkoli Anbarani², Rômulo Meira-Góes³, and Ilya Kovalenko^{1,2}

Abstract—Manufacturing industries are facing increasing product variability due to the growing demand for personalized products. Under these conditions, ensuring safety becomes challenging as frequent reconfigurations can lead to unintended hazardous behaviors. Multi-agent control architectures have been proposed to improve flexibility through decentralized decision-making and coordination. However, these architectures are based on predefined task models, which limit their ability to adapt task planning to new product requirements while preserving safety. Recently, large language models have been introduced into manufacturing systems to enhance adaptability, but reliability remains a key challenge. To address this issue, we propose a control architecture that leverages the flexibility of large language models while preserving safety on the manufacturing shop floor. Specifically, the proposed framework verifies large language model-enabled task allocations by using temporal logic and discrete event systems. The effectiveness of the proposed framework is demonstrated through a case study that involves a multi-robot assembly scenario, showing that unsafe tasks can be allocated safely before task execution.

I. INTRODUCTION

Manufacturing systems need to accommodate higher product variety and dynamic operating conditions due to increasing demand for personalized products [1]. These trends increase the need for control strategies that support adaptation of the production system while maintaining reliability [2]. To address these challenges, Multi-Agent System (MAS) has been proposed as a manufacturing control paradigm that distributes task execution and resource coordination through interactions among agents representing products and resources [3], [4]. MASs have been extended by formalizing agent behaviors and interactions using Discrete Event System (DES) models for structured representation of task execution and resource coordination [5], [6]. These works enable agents to dynamically explore the environment, apply optimization-based planning, and negotiate constraints for adaptive scheduling. These approaches provide structured models for adaptive decision-making, however, they rely on predefined models and decision-making components. They have limited ability to adapt to unforeseen product requirements.

Large Language Models (LLMs) have been explored to enhance flexibility in manufacturing systems by enabling adaptive planning and coordination across heterogeneous manufacturing resources [7]. For example, LLMs can support

Natural Language (NL)-driven task generation and coordination in MASs [8]. LLMs enable resource capability exploration in response to unexpected disruptions, maximizing resource capabilities at runtime [9]. While these works indicate that LLMs are effective in improving flexibility beyond predefined scenarios, their outputs are not guaranteed to satisfy task-level safety constraints, such as precedence relationships. With manufacturing systems increasingly relying on automated software, ensuring correctness is critical since software faults or incorrect state interpretations can lead to hazardous results [10], [11]. These considerations motivate the use of verification in LLM-enabled MAS in manufacturing that can improve flexible decision-making while preserving safety.

One approach for verifying automated decision-making in manufacturing is Temporal Logic (TL). TL provides a formal language for specifying temporal constraints on the ordering of events [12]. In prior studies, TL has been used to verify the behavior of agents against task specifications. Formalizing NL into TL enables robot planning and execution under safety constraints [13], [14]. However, these approaches focus on planning-level TL and do not address verification of generated plans using DES, which capture execution-level behaviors. DES enables explicit representation of events and states, allowing concurrency and resource conflicts to be identified and checked against safety constraints. Other works have explored LLM-enabled decision-making using TL as an external verification layer. For example, TL-based planners have been combined with LLMs to guide task decomposition while providing formal guarantees [15]. Similarly, constraint-based frameworks enforce safety rules to mitigate errors from language-based reasoning [16]. **While these studies demonstrate that TL can constrain LLM-driven decision-making, they do not address verification in manufacturing settings involving interacting resources, parts, and execution constraints. In these settings, safety is determined by system-level execution under shared-resource, synchronization, and ordering requirements. Consequently, even when individual task assignments are feasible for a single agent, the execution may still be unsafe due to resource conflicts and precedence violations. This motivates the use of DES-based verification.**

R1.1

In this study, we present a control architecture that enables the verification of LLM-generated task plans in multi-agent manufacturing systems. The proposed framework is inspired by Counterexample Guided Inductive Synthesis (CEGIS) [17], where candidate solutions are iteratively checked and revised based on formally detected violations. In manufactur-

The authors [1] are with the Department of Industrial and Manufacturing Engineering, [2] are with the Department of Mechanical Engineering, and [3] are with the Department of Electrical Engineering, at the Pennsylvania State University, State College, USA (e-mail: {jx1567; mkt5457; romulo; iqk5135}@psu.edu).

ing, new product requirements are often difficult to represent within predefined models that were not originally designed to accommodate such variability. While CEGIS focuses on bounded synthesis, we utilize LLM to introduce new product requirements by composing resource capabilities into multi-agent tasks through reasoning over requirement-capability relationships. DES provides execution-level details for these plans, enabling them to be formally evaluated. This way, logic-based verification ensures correctness without redesigning the models. The main contributions of this work are: (1) a control architecture that supports verification of LLM-generated task plans, (2) a verification process that evaluates task plans with TL represented as automata, and (3) a feedback mechanism that utilizes verification results to refine task plans. The rest of this paper is organized as follows. Section II introduces the preliminaries on TL and DES. Section III describes the formulation of the architecture for verifying LLM-generated task plans. Section IV presents the TL-based verification process and its automata. Section V illustrates the approach through a case study. Finally, the conclusion is given in Section VI.

II. PRELIMINARIES

A. Finite State Automata

Finite State Automata (FSA) is a modeling framework in DES and used to express execution logic and process sequencing [18]. Manufacturing system behavior is represented using FSA to support verification of temporal safety constraints. A finite-state automaton is defined as $\mathcal{A} = (X, E, Tr, x_0, X_m)$, where X is a finite set of states, E is a set of events, $Tr : X \times E \rightarrow X$ is a transition function, $x_0 \in X$ is the initial state, and $X_m \subseteq X$ is the set of marked states corresponding to desired behaviors [18].

B. Linear Temporal Logic on Finite Traces

A major part of the safe execution of manufacturing plans is characterized by satisfaction of temporal constraints. A task plan is considered safe depending on the ordering and occurrence of tasks. Even when a task plan is logically correct, incorrect ordering or conflicting use of shared resources can propagate fault sequences. For this reason, temporal constraints must be enforced. In the proposed approach, Linear Temporal Logic on Finite Traces (LTLf) [19] is used to specify constraints. Corresponding formulas are used to evaluate over finite execution traces, composed of discrete events. To formally specify temporal constraints, we define atomic propositions that characterize observable events.

Definition 1 (Atomic Propositions): Let AP be a finite set of atomic propositions and $ap \in AP$ be an atomic proposition. In this work, each atomic proposition ap represents the observation of a manufacturing task execution event and is instantiated using the structured form:

`ap/process/product/resource/event/context`.

The structured representation follows product, process, resource modeling conventions in manufacturing ontologies [20]. This representation enables uniform specification

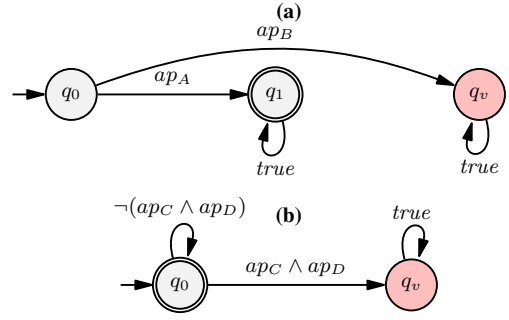


Fig. 1: Finite-state automata encoding LTLf safety constraints: (a) ordering and (b) mutual exclusion.

of task execution events. For example, an ap for an assembly pick operation that has occurred can be expressed `ap/assembly/part_A/robotarm1/pick_part/printerX`. The event field represents the type of action being observed (e.g., `pick_part`, `place_part`), while the context field captures relevant conditions, such as location. Using these ap representations, temporal safety constraints can be specified as LTLf formulas.

Definition 2 (LTLf Formula): An LTLf formula φ is constructed from atomic propositions in AP using boolean and temporal operators according to the standard LTLf syntax:

$$\varphi := \top \mid ap \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid X\varphi \mid \varphi_1 U \varphi_2,$$

where $ap \in AP$. The symbol $\top$ is the boolean constant *true*, $\neg$ and $\wedge$ is boolean negation and conjunction, respectively. X and U are the *next* and *until* temporal operators. The boolean operators $\vee$ and $\rightarrow$, as well as the temporal operators *eventually* (F) and *globally* (G), are also used. They express disjunction, implication, eventual occurrence, and conditions that must hold over the execution trace, respectively.

Each safety constraint is specified as an LTLf formula and evaluated over finite execution traces. For example, an ordering constraint can be expressed as $\varphi = \neg ap_B U ap_A$, which specifies a precedence relationship between task execution events. Another example is a mutual exclusion requirement, $\varphi = G \neg(ap_C \wedge ap_D)$, which enforces that two observable events do not occur simultaneously. These constraints can be translated into equivalent FSA for trace-based validation, as shown in Fig. 1.

Definition 3 (Safety Automaton): Each temporal safety constraint, which is specified as an LTLf formula in Definition 2, is translated into a safety automaton $\mathcal{A}_s = (Q_s, \Sigma_s, \delta_s, q_{0,s}, Q_{v,s})$, where Q_s is a finite set of safety states, Σ_s is the set of AP in the safety specification, $\delta_s : Q_s \times 2^{\Sigma_s} \rightarrow Q_s$ is the transition function, $q_{0,s} \in Q_s$ is the initial state, and $Q_{v,s} \subseteq Q_s$ is the set of violation states. A safety constraint is violated when a state in $Q_{v,s}$ is reached.

This definition formalizes the translation of LTLf-based safety constraints into automata for verification. By representing each constraint as a safety automaton with violation states, safety verification can be performed, as described in Section IV.

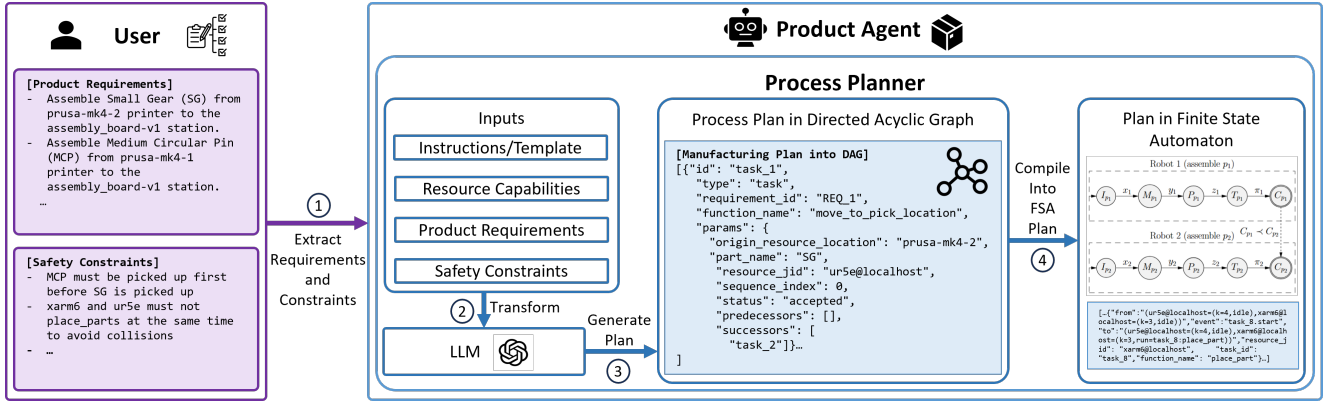


Fig. 2: Process planning workflow. NL product requirements are transformed into structured requirements using an LLM, generating a task plan in DAG, and compiling this DAG into a FSA plan.

III. AGENT AND PROBLEM FORMULATION

A. Multi-Agent Architecture Overview

Various MAS architectures introduce a Product Agent (PA) and Resource Agent (RA) structure. PAs manage processing sequences for physical parts, while RAs execute tasks on resources based on coordination with PAs [5]. In this section, we detail the roles of PA and RA and introduce the formal representations required for verifying task planning and allocation.

1) *Resource Agents*: Each RA is modeled as a FSA capturing the executable behavior of the resource [9]. The RA behavior is modeled as $A_r = (X_r, E_r, Tr_r, x_{0,r}, X_{m,r})$, where X_r is a set of physical states, E_r is a set of task execution events that corresponds to a physical actions (e.g., the start or completion of tasks), and Tr_r maps states and events to resulting states. $x_{0,r}$ is the initial state, and $X_{m,r}$ corresponds to task completion for a part.

2) *Product Agents*: A PA is responsible for realizing the production requirements of a physical product [5]. In this work, LLM-enabled PAs interpret NL product requirements and available resource capabilities to generate a task plan represented as a Directed Acyclic Graph (DAG). This representation captures task dependencies and concurrency and supports compilation into an automaton for discrete-event modeling and verification, as illustrated in Fig. 2. The automaton is formally defined as $A_{plan} = (X_{plan}, E_{plan}, Tr_{plan}, x_0, X_m)$, which models the joint behavior of all RAs required to satisfy the product requirements. Let $\mathcal{J}$ be the index set of RAs. The global state space is defined as $X_{plan} \subseteq \prod_{j \in \mathcal{J}} X_j$, where X_j denotes the set of physical states of RA j . Each global state $x \in X_{plan}$ represents the joint physical state of all resources. Let $G = (T, D)$ be the task plan DAG, where $T = \{\tau_1, \dots, \tau_N\}$ is the set of subtasks to satisfy a product requirement, and D is the precedence relation between tasks. From this task set, the event set is defined as $E_{plan} = \{\tau_i^{start}, \tau_i^{done} \mid \tau_i \in T\}$, where τ_i^{start} and τ_i^{done} represent the start and completion of task τ_i , respectively. The transition $Tr_{plan} \subseteq X_{plan} \times E_{plan} \times X_{plan}$ is derived from DAG. A transition labeled by τ_i^{start} occurs once all predecessor tasks of τ_i have completed. The initial state

represents no tasks executed, and marked states correspond to successful completion of the global task plan.

B. Problem Formulation

We consider an LLM-enabled multi-agent manufacturing system that supports adaptive task planning and allocation. Product and safety requirements are provided in NL to guide the task planning process. However, due to known limitations of LLM, the PA may misinterpret instructions during task planning that involve concurrency of resources [8]. As a result, the synthesized plan may still violate safety constraints, such as incorrect ordering of tasks or unsafe simultaneous actions. The problem addressed in this work is as follows. When new product requirements are introduced, the challenge is to generate task plans without relying on predefined task models (e.g, fixed task sequences) while satisfying all safety constraints on the shop floor.

1) *Assumptions*: To address this problem, the proposed framework operates under the following assumptions.

- A.1 The synthesized formal safety specifications accurately capture the intended safety constraints expressed in NL and are validated by a domain expert.
- A.2 Task execution events relevant to safety constraints are observable and can be mapped to atomic propositions.
- A.3 Product requirements expressed in NL are achievable with the given manufacturing capabilities in the system.

These assumptions are introduced to focus on the safety verification problem. A.1 allows the framework to focus on verifying formal constraints rather than semantic correctness. A.2 ensures that safety-relevant events can be observed within the framework. A.3 excludes infeasible planning cases that do not reflect safety-related failures. These assumptions allow the study to focus on verifying the safety properties of generated task plans.

IV. CONTROL ARCHITECTURE FOR VERIFICATION

In this section, we introduce the Central Controller Agent (CCA). Following the control architecture proposed by [21], the CCA operates with a global view of the MAS to preserve safety guarantees while supporting adaptive task allocation.

While distributed runtime verification [22] can represent **R1.5**

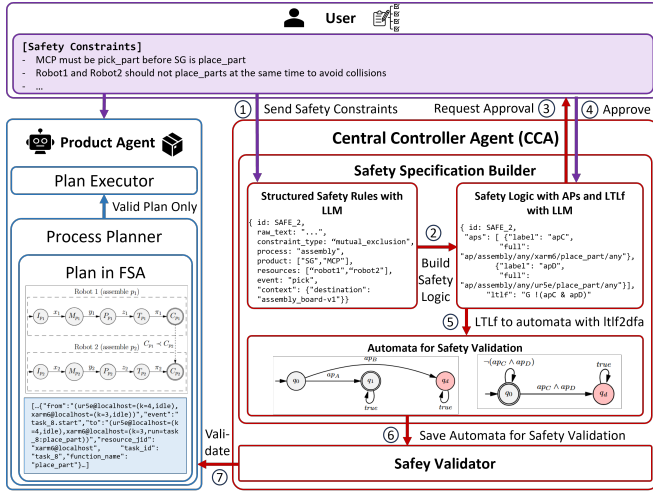


Fig. 3: Safety-aware task planning workflow. The CCA translates NL safety constraints into LTLf specifications, which are then compiled into FSA. These automata are used to validate the LLM-generated task plan before execution.

new global constraints, they require those constraints to be translated into local conditions for the affected agents. Since synthesized plans can introduce new dependencies between agents, the CCA was proposed to verify the joint plan directly.

A. Safety Specification

To enforce safety while accommodating evolving product requirements, we propose a control architecture that formalizes NL safety requirements into automata for DES-based verification, as illustrated in Fig. 3. Safety requirements are specified by a human operator with knowledge of manufacturing safety rules and operational constraints. Let $\mathcal{S} = \{s_1, \dots, s_M\}$ be the set of NL safety requirements. Each requirement $s_i \in \mathcal{S}$ is transformed into a structured safety representation that includes constraint type, associated process, product, resource, event, and contexts using LLM to map NL safety requirements into structured elements. This representation allows CCA to convert it into a corresponding set of atomic propositions $AP_i = \{ap_{i,1}, \dots, ap_{i,k_i}\} \subseteq AP$, where each $ap_{i,j}$ represents the observation of a task execution event, as formalized in Definition 1. This set of atomic propositions AP_i is then used to construct an LTLf formula $\varphi_i \in \text{LTL}_f(AP_i)$, which encodes the safety constraint s_i , as specified in Definition 2. Each formula φ_i is translated into a safety automaton $\mathcal{A}_s$ according to Definition 3.

R2.2 Since the translation of NL safety requirements into formal specifications is not guaranteed, new safety constraints require expert validation [16]. Therefore, based on assumption A.1, the synthesized LTLf and corresponding $\mathcal{A}_s$ are validated by a domain expert to correctly capture the intended safety constraints. As this manual validation limits scalability in highly dynamic environments, future work will explore methods such as fine-tuning to reduce reliance on expert intervention.

Algorithm 1: DFS-Based Safety Validation

```

1: procedure VALIDATESAFETY( $\mathcal{A}_{plan}, \mathcal{A}_{safety}$ )
2:    $(X_{plan}, E, Tr, x_0, X_m) \leftarrow \mathcal{A}_{plan}$ 
3:   for each safety automaton  $\mathcal{A}_s \in \mathcal{A}_{safety}$  do
4:      $\mathcal{A}_s = (Q_s, \Sigma_s, \delta_s, q_0, s, Q_{v,s})$ 
5:      $R \leftarrow \{(x_0, q_0, s)\}$   $\triangleright$  reachable product states
6:      $V \leftarrow \emptyset$   $\triangleright$  visited product states
7:     while  $R \neq \emptyset$  do
8:        $(x, q) \leftarrow \text{Pop}(R)$ 
9:       if  $(x, q) \in V$  then continue
10:       $V \leftarrow V \cup \{(x, q)\}$ 
11:      if  $x \in X_m$  then
12:         $q_{end} \leftarrow \delta_s(q, \emptyset)$ 
13:        if  $q_{end} \in Q_{v,s}$  then
14:          return Violation Found
15:      for each transition  $(x, e, x') \in Tr$  from  $x$  do
16:         $\sigma \leftarrow \text{EXTRACTAPS}(x')$ 
17:         $q' \leftarrow \delta_s(q, \sigma)$ 
18:        if  $q' \in Q_{v,s}$  then
19:          return Violation Found
20:       $R \leftarrow R \cup \{(x', q')\}$ 
21:   return Safe

```

B. Safety Validation

After the CCA constructs safety logic from user-defined constraints, it performs safety validation using a reachability analysis. This analysis is conducted on the product of the plan execution automaton $\mathcal{A}_{plan}$ and a set of safety automata $\mathcal{A}_{safety}$ converted from LTLf-based temporal safety specifications. The goal is to determine whether any task plan in an automata violates a safety constraint before execution. The safety validation workflow, performed by CCA, is illustrated in Fig. 4.

Safety validation begins when a PA submits a compiled plan execution automaton $\mathcal{A}_{plan}$ to the CCA. The automaton captures the joint execution states of all resources in processing a product. Given $\mathcal{A}_{plan}$ and a set of safety automata $\mathcal{A}_{safety}$, Algorithm 1 details the reachability analysis performed over product states (x, q) .

Starting from the initial product state (x_0, q_0, s) , the CCA performs a Depth First Search (DFS)-based reachability analysis over product states (x, q) . For each plan transition $(x, e, x') \in Tr$, safety atomic propositions are evaluated at the resulting state x' . These propositions are used as input to the safety automaton, which transitions to $q' = \delta_s(q, \sigma(x'))$. If $q' \in Q_{v,s}$, a safety violation is reported. When the reachability analysis ends without reaching any violation state $q_{violation}$, including marked plan states $x \in X_m$, the plan is considered safe. All detected safety violations are accumulated by the CCA and used as feedback for replanning via PA, as described in Section IV-C.

C. Replanning with Feedback

For replanning, the PA receives a structured feedback prompt that frames plan repair as a grounded constrained-generation problem, as shown in Table I. The prompt includes a diagnosis of the failure state. This diagnosis contains the unsafe task plan, the violated safety condition, and a witness trace showing how the violation occurs. It also provides the constraints for generating a valid repair. These include repair instructions, execution context, and a machine-

R2.1

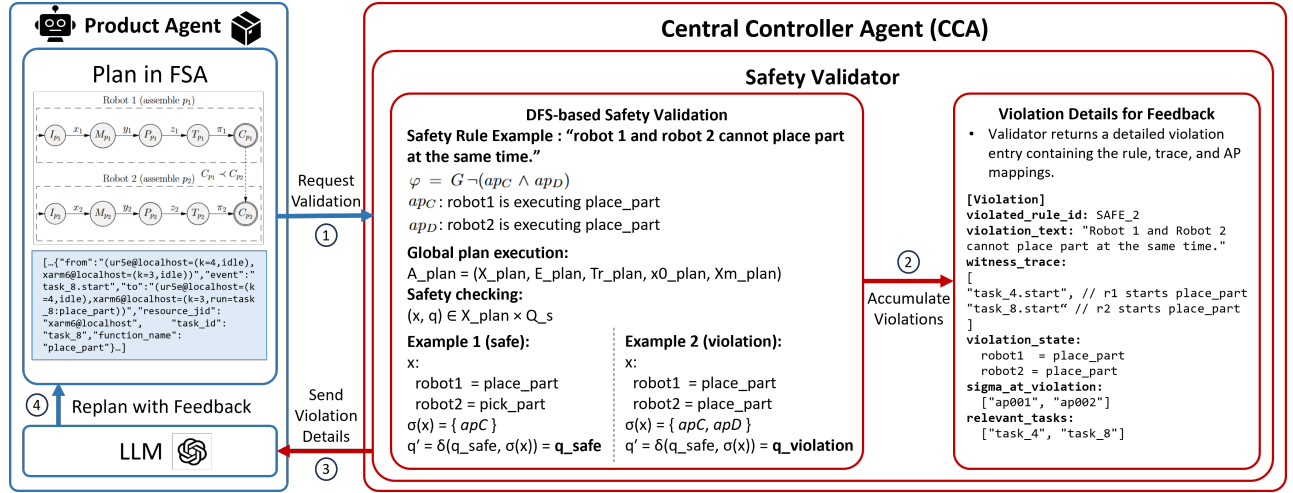


Fig. 4: CCA safety validation architecture. The task plan is traversed with the safety automata using DFS-based validation to detect violations. Safety violations are accumulated and returned as structured feedback for replanning.

TABLE I: Representative examples of the feedback prompt.

Prompt section	Representative examples
System instructions	You are a plan expert in pre-execution mode. Fix the unsafe plan before execution begins. ...
Unsafe plan	{id: SG_MOVE, function: move_loaded, part: SG, resource_jid: xarm6, predecessors: [SG_PICK], ...}
Safety violation	MCP_MOVE must occur before SG_MOVE. ...
Violation trace	witness_events = [SG_MOVE.start, ...]; the trace shows SG_MOVE begins before the required MCP_MOVE.
Execution context	functions = [pick_part, move_loaded, ...]; agents = [xarm6, ur5e]; locations = [prusa-mk3, assembly_board, ...]
Output schema	{tasks: [{id: TASK_ID, predecessors: [TASK_ID], change_reason: explanation of the repair}]}

readable output schema. This structure helps the model identify what must change and return a feasible plan update that the system can apply directly.

V. CASE STUDY

A. Case Study Setup

To evaluate the proposed architecture, an assembly scenario is set up. The physical setup, shown in Fig. 5(a), consists of two robot arms (xarm6 and ur5e), operating in a workspace with a picking location (prusa-mk3) and an assembly location (assembly_board-v1). A human operator is responsible for assembly and safety requirements.

The system receives NL assembly requirements provided by a human operator, specifying that a small gear (SG) and a medium circular pin (MCP) must be assembled from the picking location to the assembly board. Given a predefined set of robot capabilities (e.g., pick_part), a PA interprets these requirements and dynamically generates a task plan, including the allocation of tasks. Two NL safety constraints $\mathcal{S}$ are also provided by a human operator. The first constraint specifies that xarm6 and ur5e must not execute pick_part simultaneously. The second constraint requires that ur5e must execute move_loaded_to_destination before xarm6. The objective is to assess whether the proposed framework

TABLE II: List of task execution states and events.

Symbol	Robot Execution Status / Event
$ISG, IMCP$	Idle
M_{SG}, M_{MCP}	Moving to the picking location
P_{SG}, P_{MCP}	Picking part
T_{SG}, T_{MCP}	Moving part to the assembly location
C_{SG}, C_{MCP}	Placing part
x_1, x_2	Move to pick location
y_1, y_2	Pick part
z_1, z_2	Move loaded part to destination
π_1, π_2	Place part

can synthesize task plans that satisfy these safety constraints without predefining task allocations. GPT-4o by OpenAI was utilized in this case study [23].

B. Case Study and Task Planning Results

As introduced in Section III-A.2, the PA synthesizes a task plan from the NL requirements and compiles it into A_{plan} . The execution states and events used are defined in Table II. Although safety requirements $\mathcal{S}$ are provided to the PA to guide task allocation, the initial task plan synthesized may still violate $\mathcal{S}$ in concurrent multi-agent settings, as shown in Fig. 5(b). Specifically, the PA does not explicitly enforce required task ordering relations during planning, misinterpreting the constraints with LLM. This leads to unsafe concurrent execution when safety constraints are incompletely incorporated into the generated plan. We utilize the control architecture introduced in Section IV to enforce and verify these safety-critical actions, after the synthesized LTLf safety specifications are validated by a domain expert. Consider the first constraint on the mutual exclusion, preventing simultaneous execution is specified as $\varphi_{pick} = G \neg (ap_{pick}^{xarm6} \wedge ap_{pick}^{ur5e})$, where ap_{pick}^{xarm6} and ap_{pick}^{ur5e} are the execution of the pick_part events leading to states P_{SG} and P_{MCP} , respectively. During safety validation, we utilize the DFS-based reachability analysis described in Algorithm 1. If the plan automaton reaches a state $x \in$

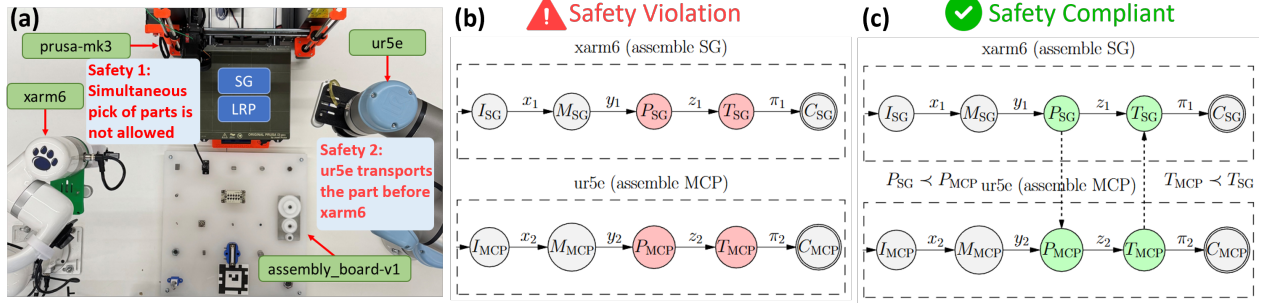


Fig. 5: Case study setup and task planning results. (a) Multi-robot assembly setup, where two arms assemble parts on a shared assembly board, (b) Initial task planning automata generated by the PA without safety validation, where pick_part and move_loaded_to_destination stages violate safety constraints, and (c) The task planning automata after safety validation and repair, where synthesized precedence constraints are enforced.

X_{plan} such that $q' = \delta_s(q, \{ap_{\text{pick}}^{\text{xarm6}}, ap_{\text{pick}}^{\text{ur5e}}\})$, the safety automaton enters a violation state $q' \in Q_{v,s}$. Similarly, the ordering constraint on moving parts to the assembly location is specified as $\varphi_{\text{move}} = \neg ap_{\text{move}}^{\text{xarm6}} \cup ap_{\text{move}}^{\text{ur5e}}$, where $ap_{\text{move}}^{\text{xarm6}}$ and $ap_{\text{move}}^{\text{ur5e}}$ are the transport events z_1 and z_2 for parts SG and MCP, respectively. The safety automaton $\mathcal{A}_s$ encodes this precedence by transitioning to a violation state if $ap_{\text{move}}^{\text{xarm6}}$ is observed before $ap_{\text{move}}^{\text{ur5e}}$. During safety validation, if the plan automaton reaches a state $x \in X_{\text{plan}}$ such that $ap_{\text{move}}^{\text{xarm6}} \in \sigma(x)$ and $ap_{\text{move}}^{\text{ur5e}} \notin \sigma(x_i)$ for all preceding states x_i , the safety automaton transition is $q' = \delta_s(q, \{ap_{\text{move}}^{\text{xarm6}}\})$, $q' \in Q_{v,s}$. The CCA returns violation feedback to the PA with the information described in Section IV-C. The PA receives the violated constraint, the execution trace, and prompt instructions. Using this information, the PA introduces additional precedence constraints in the task DAG to prevent the unsafe behavior. The repaired task plan is shown in Fig. 5(c), where the ordering constraints enforced from replanning process $P_{SG} \prec P_{MCP}$, $T_{MCP} \prec T_{SG}$ satisfy the given safety constraints $\mathcal{S}$. This case study demonstrates that the proposed framework can identify and repair safety violations in LLM-generated task plans.

VI. CONCLUSION

In this work, we present a control architecture for verifying task allocation in LLM-enabled multi-agent manufacturing systems using TL and DES. Specifically, the proposed framework utilizes an LLM-enabled PA to dynamically generate a task plan with a CCA that enforces safety constraints through automata-based verification. By structuring NL safety constraints into LTLf specifications and corresponding FSA, the architecture validates the LLM-generated task plans before task execution. An assembly case study scenario demonstrates that unsafe task allocations can be formally detected and repaired before execution, without limiting planning flexibility. Future work will focus on improving the robustness and generalizability of the proposed verification framework across diverse multi-agent manufacturing scenarios with various tasks, resources, and safety constraints. **As the framework scales to larger manufacturing plants, a single CCA can become impractical, motivating the investigation of subsystem-level or hierarchical verification architectures to**

reduce communication and computation bottlenecks. The framework will also be extended to enable online safety verification to support runtime adaptation when unexpected events occur while preserving temporal safety constraints.

REFERENCES

- [1] X. Gu and Y. Koren, "Smart factories for mass individualization," *Encyclopedia*, vol. 4, no. 1, pp. 415–429, 2024.
- [2] B. Vogel-Heuser, A. Fay, I. Schaefer, and M. Tichy, "Evolution of software in automated production systems: Challenges and research directions," *Journal of Systems and Software*, vol. 110, pp. 54–84, 2015.
- [3] J. Barbosa, P. Leitão, E. Adam, and D. Trentesaux, "Dynamic self-organization in holonic multi-agent manufacturing systems: The adacor evolution," *Computers in Industry*, vol. 66, pp. 99–111, 2015.
- [4] I. Kovalenko, D. Tilbury, and K. Barton, "Priced timed automata models for control of intelligent product agents in manufacturing systems," in *Proceedings of the 15th IFAC Workshop on Discrete Event Systems (WODES)*, 2020, pp. 136–142.
- [5] —, "The model-based product agent: A control oriented architecture for intelligent products in multi-agent manufacturing systems," *Control Engineering Practice*, vol. 86, pp. 105–117, 2019.
- [6] I. Kovalenko, E. C. Balta, D. M. Tilbury, and K. Barton, "Cooperative product agents to improve manufacturing system flexibility: A model-based decision framework," *IEEE Transactions on Automation Science and Engineering*, vol. 20, no. 1, pp. 440–457, 2022.
- [7] S. Zhao, S. Liu, Y. Jiang, B. Zhao, Y. Lv, J. Zhang, L. Wang, and R. Y. Zhong, "Industrial foundation models (ifms) for intelligent manufacturing: A systematic review," *Journal of Manufacturing Systems*, vol. 82, pp. 420–448, 2025.
- [8] J. Lim, B. Vogel-Heuser, and I. Kovalenko, "Large language model-enabled multi-agent manufacturing systems," in *2024 IEEE 20th International Conference on Automation Science and Engineering (CASE)*. IEEE, 2024, pp. 3940–3946.
- [9] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *2025 IEEE 21st International Conference on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095.
- [10] D. Schütz, A. Wannagat, C. Legat, and B. Vogel-Heuser, "Development of plc-based software for increasing the dependability of production automation systems," *IEEE Transactions on Industrial Informatics*, vol. 9, no. 4, pp. 2397–2406, 2013.
- [11] M. T. Anbarani, J. Lim, R. Meira-Góes, and I. Kovalenko, "Modeling and mitigating silent faults in coupled discrete event systems," in *Proceedings of the 17th IFAC Workshop on Discrete Event Systems (WODES)*, 2024, pp. 168–173.
- [12] A. Pnueli, "The temporal logic of programs," in *18th Annual Symposium on Foundations of Computer Science (sfcs 1977)*. IEEE, 1977, pp. 46–57.
- [13] J. X. Liu, Z. Yang, B. Schornstein, S. Liang, I. Idrees, S. Tellex, and A. Shah, "Lang2ltl: Translating natural language commands to temporal specification with large language models," in *Workshop on Language and Robotics at CoRL 2022*, 2022.

- [14] S. Xu, X. Luo, Y. Huang, L. Leng, R. Liu, and C. Liu, “NI2hltl2plan: Scaling up natural language understanding for multi-robots through hierarchical temporal logic task specifications,” *IEEE Robotics and Automation Letters*, vol. 10, no. 10, pp. 10 482–10 489, 2025.
- [15] J. Wang, J. Tong, K. Tan, Y. Vorobeychik, and Y. Kantaros, “Conformal temporal logic planning using large language models,” *ACM Transactions on Cyber-Physical Systems*, Sep. 2025.
- [16] Z. Yang, S. S. Raman, A. Shah, and S. Tellex, “Plug in the safety chip: Enforcing constraints for llm-driven robot agents,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 14 435–14 442.
- [17] A. Solar-Lezama, *Program Synthesis by Sketching*. University of California, Berkeley, 2008.
- [18] C. G. Cassandras and S. Lafortune, *Introduction to Discrete Event Systems*, 3rd ed. Springer, Cham, 2021.
- [19] G. De Giacomo and M. Y. Vardi, “Linear temporal logic and linear dynamic logic on finite traces.” in *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI)*, vol. 13, 2013, pp. 854–860.
- [20] S. Lemaignan, A. Siadat, J.-Y. Dantan, and A. Semenenko, “Mason: A proposal for an ontology of manufacturing domain,” in *IEEE Workshop on Distributed Intelligent Systems: Collective Intelligence and Its Applications (DIS’06)*. IEEE, 2006, pp. 195–200.
- [21] I. Kovalenko, J. Moyne, M. Bi, E. C. Balta, W. Ma, Y. Qamsane, X. Zhu, Z. M. Mao, D. M. Tilbury, and K. Barton, “Toward an automated learning control architecture for cyber-physical manufacturing systems,” *IEEE Access*, vol. 10, pp. 38 755–38 773, 2022.
- [22] A. Francalanza, J. A. Pérez, and C. Sánchez, “Runtime verification for decentralised and distributed systems,” *Lectures on Runtime Verification: Introductory and Advanced Topics*, pp. 176–210, 2018.
- [23] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, “Gpt-4o system card,” *arXiv preprint arXiv:2410.21276*, 2024.